

SCSS

INFO

Lesinhoud

Je leert wat SCSS is, hoe je het installeert en gebruikt in een vanilla project, en wat de belangrijkste concepten zijn zoals variabelen, nested rules, mixins, modules en imports, functies en iteraties. Je leert ook kort hoe je SCSS kan gebruiken in Vue-componenten.

Doelstellingen

De student(e) kan:

- Werken met een Command Line Interface (CLI) / Node Package Manager (NPM) voor het installeren, updaten en verwijderen van pakketten en modules binnen een project.
- SCSS schrijven en vervolgens omzetten naar CSS.

▼ Inhoud

- [Wat is SCSS?](#)
- [Installeren van SCSS](#)
- [SCSS compileren](#)
- [Automatisch compileren](#)
- [Variabelen](#)
- [Nested Rules](#)
- [Mixins](#)
- [Modules en Imports](#)

- [Iteraties en Functies](#)
- [SCSS en Vue](#)
- [Conclusie](#)

Wat is SCSS?

SCSS is een **CSS preprocessor**. Dit betekent dat je SCSS-code omzet naar gewone CSS-code die browsers begrijpen. SCSS voegt extra functionaliteiten toe aan CSS, zoals variabelen, nested rules, mixins, functies, ...

Voordat je SCSS koppelt aan Vue, leer je eerst hoe SCSS op zichzelf werkt. Hiervoor gebruik je NPM om SCSS te installeren en een project aan te maken.

Er zijn geen aparte oefeningen voor dit onderdeel. De theorie bevat de bouwstenen om SCSS te gebruiken. Volg deze stappen om SCSS te leren kennen.

Installeren van SCSS

Voer alle commando's uit in een terminal vanuit de **root** van het project.

Maak een nieuw NPM-project aan in de werkmap.

```
npm init -y
```

sh

Installeer vervolgens SCSS als **dev dependency**.

```
npm install sass --save-dev
```

sh

dev dependency

Wanneer je software via NPM installeert, kies je of je deze als **dependency** of **dev dependency** wilt installeren. Een dependency is nodig opdat de applicatie die je bouwt in productie werkt. Een dev dependency heb je enkel nodig tijdens de ontwikkeling van de software. SCSS heb je enkel nodig tijdens de ontwikkeling, omdat je het omzet naar gewone CSS die de browser begrijpt.

Bij het installeren kies je of je de dependency in productie nodig hebt of enkel tijdens de ontwikkeling.

▼ SCSS/SASS

SCSS is een syntactische variant van SASS. SASS gebruikt inspringingen in plaats van accolades en puntkomma's. SCSS is meer vergelijkbaar met gewone CSS, maar met extra functionaliteiten. In deze les wordt SCSS gebruikt.

Voorbeeld SASS

```
$primary-color: #663399
body
  color: $primary-color
```

SCSS

Voorbeeld SCSS

```
$primary-color: #663399;
body {
  color: $primary-color;
}
```

SCSS

Dit wordt in beide gevallen omgezet naar:

```
body {
  color: #663399;
```

CSS

```
}
```

CONVENTIE

In dit opleidingsonderdeel worden SCSS-bestanden gebruikt, met de extensie

`.SCSS`.

SCSS compileren

Om SCSS-code om te zetten naar gewone CSS-code, moet je de SCSS-bestanden compileren. Dit doe je via de command line interface (CLI) met het `sass`-commando. Je installeert `sass` als NPM-pakket in het project. Wanneer je in een `npm script` naar `sass` verwijst, controleert NPM of het pakket in de `node_modules`-map van het project staat. Zo ja, dan gebruikt NPM dat pakket om het commando uit te voeren.

Voeg een script toe aan `package.json` om SCSS te compileren.

```
// hier nog code
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "compile": "sass scss/input.scss css/output.css"
},
// hier nog code
```

json

Je verwijst naar een bestand `input.scss` in de `scss`-map. Maak deze map en het bestand aan.

```
mkdir scss
```

sh

```
touch scss/input.scss
```

sh

Je geeft aan dat het CSS-bestand dat je compileert `output.css` zal heten en je het in de `css` -map plaatst. Het systeem maakt deze map automatisch aan wanneer je het `sass` -commando uitvoert.

Voeg onderstaande SCSS-code toe aan `scss/input.scss` .

```
@use "sass:color";

$primary-color: #4f81bd;
$secondary-color: #2ecc71;

body {
  font-family: Arial, sans-serif;
  background-color: $primary-color;
  color: white;
}

button {
  background-color: $secondary-color;
  border: none;
  padding: 10px 20px;
  color: white;
  cursor: pointer;

  &:hover {
    background-color: color.adjust($secondary-color,
    $lightness: -10%);
  }
}
```

SCSS

Heel wat code lijkt op gewone CSS, maar je ziet ook enkele nieuwe onderdelen:

- `$primary-color` en `$secondary-color` zijn variabelen. Deze kunnen hergebruikt worden in de code (zoals bij de `background-color` van `body` en `button`).
- `&:hover` is een nested rule. Dit betekent dat de `:hover` psuedoclass toegepast wordt op het element waar `button` op toegepast wordt. Dit is hetzelfde als `button:hover` in gewone CSS.
- `@use "sass:color";` importeert de ingebouwde `color`-module van SCSS, zodat functies zoals `color.adjust()` gebruikt kunnen worden om kleuren aan te passen.
- `color.adjust($secondary-color, $lightness: -10%)` past de lichtheid van de kleur aan, waardoor de hover-kleur iets donkerder wordt dan de originele secundaire kleur.

Voer nu het compileerscript uit via NPM.

```
npm run compile
```

sh

Dit zet het SCSS-bestand `scss/input.scss` om naar het CSS-bestand `css/output.css`. Bekijk de inhoud van `css/output.css` om het resultaat te zien. Het systeem maakt ook een `css/output.css.map`-bestand aan. Dit is een `source map` die je kan gebruiken met ontwikkeltools in browsers om de gegenereerde CSS terug te koppelen naar de originele SCSS-code voor debuggingdoeleinden.

Je ziet dat het systeem de variabelen naar hun respectievelijke waarden wijzigt, de nested rules naar gewone CSS-selectors omzet en de kleurfunctie naar de juiste kleurwaarde omzet.

Dit is het bestand (`css/output.css`) dat je in onze HTML-pagina's gebruikt om de stijlen toe te passen.

Voeg een HTML-bestand toe om de CSS te gebruiken.

```
touch index.html
```

sh

html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Document</title>
    <link rel="stylesheet" href="./css/output.css" />
  </head>
  <body>
    <button class="button-primary">Primary</button>
    <button class="button-secondary">Secondary</button>
    <button class="button-danger">Danger</button>
  </body>
</html>
```

Automatisch compileren

Je leert wat SCSS is, hoe je het installeert en gebruikt in een vanilla project, en wat de belangrijkste concepten zijn zoals variabelen, nested rules, mixins, modules en imports, functies en iteraties. Je leert ook kort hoe je SCSS gebruikt in Vue-componenten. Momenteel moet je continu manueel `npm run compile` uitvoeren om de SCSS te compileren naar CSS wanneer je wijzigingen aanbrengt in het SCSS-bestand. Omdat je het CSS-bestand in je HTML-pagina gebruikt, is het belangrijk dat dit bestand steeds up-to-date is met de laatste wijzigingen in het SCSS-bestand.

Om te voorkomen dat je dit steeds manueel moet doen, voeg je het `--watch` -attribuut toe aan het `sass` -commando in het `compile` -script in `package.json`.

```
// hier nog code
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "compile": "sass scss/input.scss css/output.css",
  "dev": "sass --watch scss/input.scss css/output.css"
},
// hier nog code
```

json

Bij het uitvoeren van `npm run dev` zet het systeem het `sass` -commando in `watch` -modus. Dit betekent dat elke keer wanneer het systeem een wijziging in `scss/input.scss` detecteert, het het bestand automatisch opnieuw naar `css/output.css` compileert.

```
npm run dev
```

sh

Let op

Zolang het `sass --watch` -commando actief is in de terminal, kan je deze terminal niet voor andere commando's gebruiken. Open een tweede terminalvenster om andere commando's uit te voeren in hetzelfde project.

Om het `sass --watch` -commando te stoppen, typ je `CTRL + C` in de terminal.

Wijzig iets in `scss/input.scss` en sla het bestand op. Bekijk vervolgens `css/output.css` om te zien dat het systeem de wijzigingen automatisch heeft doorgevoerd.

Variabelen

Variabelen in SCSS definieer je met een `$` -teken gevolgd door de naam van de variabele. Je kan ze gebruiken om waarden op te slaan die je in

de stylesheet opnieuw kan gebruiken, zoals kleuren, lettergroottes, marges, etc.

Test uit door `scss/input.scss` te wijzigen naar onderstaande code. Bekijk vervolgens het resultaat in `css/output.css` na het opslaan van het bestand.

```
SCSS

$primary-color: #4f81bd;
$secondary-color: #2ecc71;
$danger-color: #e74c3c;
$background-color: #333333;
$text-color: #f5f5f5;
$font-size-base: 16px;

body {
  font-size: $font-size-base;
  background-color: $background-color;
  color: $text-color;
}

button {
  background-color: $primary-color;
  color: white;

  &:hover {
    background-color: $secondary-color;
  }
}
```

Vanilla CSS

Er bestaan tegenwoordig ook CSS-variabelen in gewone CSS. Dit noemt `CSS Custom Properties`. Het gebruik ervan is iets anders dan SCSS-variabelen, maar ze dienen een gelijkaardig doel.

```
CSS

:root {
  --primary-color: #4f81bd;
```

```

--secondary-color: #2ecc71;
--danger-color: #e74c3c;
--background-color: #333333;
--text-color: #f5f5f5;
--font-size-base: 16px;
}

body {
  font-size: var(--font-size-base);
  background-color: var(--background-color);
  color: var(--text-color);
}

button {
  background-color: var(--primary-color);
  color: white;
}

button:hover {
  background-color: var(--secondary-color);
}

```

Het grote verschil is dat je CSS-variabelen dynamisch kan aanpassen (in `runtime`), terwijl SCSS-variabelen tijdens het compileren door hun waarden vervangt - je kan ze niet meer aanpassen nadat je de CSS gegenereerd hebt.

Nested Rules

Nested rules in SCSS maken het mogelijk om CSS-selectors binnen andere selectors te nesten. Dit helpt om de hiërarchie en relatie tussen elementen duidelijker te maken en vermindert de noodzaak voor herhaalde selectors.

```

$primary-color: #4f81bd;
$secondary-color: #2ecc71;
$danger-color: #e74c3c;
$background-color: #333333;

```

SCSS

```

$text-color: #f5f5f5;
$font-size-base: 16px;

body {
  font-size: $font-size-base;
  background-color: $background-color;
  color: $text-color;
}

button {
  background-color: $primary-color;
  color: white;

  &:hover {
    background-color: $secondary-color;
  }
}

```

`button` is de hoofdselector, en `&:hover` is een geneste regel die verwijst naar de hover-toestand van het `button`-element. Je gebruikt het `&`-teken om naar de oudersselector (`button` in dit geval) te verwijzen. In CSS zet je dit om naar `button:hover`.

Vanilla CSS

In Vanilla CSS (gewone CSS) ondersteun je deze syntax tegenwoordig ook.

```

button {
  background-color: #4f81bd;
  color: white;

  &:hover {
    background-color: #2ecc71;
  }
}

```

CSS

Mixins

Mixins zijn herbruikbare blokken van SCSS-code die je kan gebruiken en vervolgens in andere delen van de stylesheet invoegen. Ze zijn handig voor het vermijden van codeduplicatie en het toepassen van consistente stijlen.

De syntax zal bekend voorkomen, het lijkt op een functie van JavaScript.

Test uit door `scss/input.scss` te wijzigen naar onderstaande code. Bekijk vervolgens het resultaat in `css/output.css` na het opslaan van het bestand.

```
SCSS

$primary-color: #4f81bd;
$secondary-color: #2ecc71;
$danger-color: #e74c3c;

@mixin button-styles($bg-color, $text-color) {
  background-color: $bg-color;
  color: $text-color;
  border: none;
  padding: 10px 20px;
  cursor: pointer;
}

.button-primary {
  @include button-styles($primary-color, white); //
  Gebruik van mixin

  &:hover {
    background-color: $secondary-color;
  }
}

.button-secondary {
  @include button-styles($secondary-color, white); //
```

Gebruik van mixin

```
&:hover {  
  background-color: $primary-color;  
}  
}
```

Modules en Imports

SCSS ondersteunt het concept van modules en imports, waardoor je de stylesheets in kleinere, makkelijker te beheren bestanden kan opdelen. Dit helpt bij het organiseren van de code en het hergebruiken van stijlen.

Je kan een nieuw bestand aanmaken, bijvoorbeeld

`scss/_variables.scss`, waarin je de variabelen definieert.

```
// scss/_variables.scss  
$primary-color: #4f81bd;  
$secondary-color: #2ecc71;  
$danger-color: #e74c3c;  
$background-color: #333333;  
$text-color: #f5f5f5;  
$font-size-base: 16px;
```

SCSS

Vervolgens importeer je dit bestand in het hoofd SCSS-bestand (`scss/input.scss`) met de `@use`-regel.

```
@use "variables"; // Importeer eigen bestand  
@use "sass:color"; // Importeer ingebouwde module met  
kleurfuncties  
  
body {  
  font-size: variables.$font-size-base;
```

SCSS

```

background-color: variables.$background-color;
color: variables.$text-color;
}

@mixin button-styles($bg-color, $text-color) {
  background-color: $bg-color;
  color: $text-color;
  border: none;
  padding: 10px 20px;
  cursor: pointer;
}

.button-primary {
  @include button-styles(variables.$primary-color,
white);

  &:hover {
    background-color: color.adjust(variables.$primary-
color, $lightness: -10%);
  }
}

.button-secondary {
  @include button-styles(variables.$secondary-color,
white);

  &:hover {
    background-color: color.adjust(
      variables.$secondary-color,
      $lightness: -10%
    );
  }
}

```

Merk op dat wanneer je een variabele uit een geïmporteerd bestand gebruikt, je de bestandsnaam als `namespace` moet gebruiken, gevolgd door een punt en de variabelenaam (bv. `variables.$primary-color`).

`color.adjust()` is een functie uit de ingebouwde `color` -modules van SCSS, deze is beschikbaar nadat `@use "sass:color";` is toegevoegd.

Iteraties en Functies

SCSS biedt verschillende ingebouwde functies die je kan gebruiken om waarden te manipuleren, zoals kleurfuncties, stringfuncties, numerieke functies, enzovoort. Deze functies kunnen helpen bij het dynamisch genereren van stijlen op basis van bepaalde parameters.

Ook biedt SCSS de mogelijkheid om iteraties uit te voeren met behulp van `@for` -, `@each` - en `@while` -directives. Hiermee genereer je herhalende patronen zonder handmatig elke regel te moeten schrijven.

Test uit door `scss/input.scss` te wijzigen naar onderstaande code. Bekijk vervolgens het resultaat in `css/output.css` na het opslaan van het bestand.

Ga ervan uit dat het `variables.scss` -bestand nog steeds bestaat en je dit bestand importeert.

```
SCSS

@use "variables";
@use "sass:color";

$colors: (
  primary: variables.$primary-color,
  secondary: variables.$secondary-color,
  danger: variables.$danger-color,
);

@each $name, $color in $colors {
  .button-#{$name} {
    background-color: $color;
    color: white;
  }
}
```

```

border: none;
padding: 10px 20px;
cursor: pointer;

&:hover {
  background-color: color.adjust($color, $lightness:
-10%);
}
}
}

```

Bekijk het resultaat in `css/output.css`. Je ziet dat het systeem automatisch `.button-primary`, `.button-secondary` en `.button-danger`-klassen met de respectievelijke kleuren genereert.

SCSS en Vue

In Vue kan je SCSS gebruiken zonder de extra configuratie die hierboven handmatig moest gebeuren. Wanneer je het `lang`-attribuut aan de `<style>`-tag in een Vue-component toevoegt, gebruikt Vue automatisch de juiste preprocessor om naar gewone CSS te compileren.

NIET doen

In een Vue-project is het **niet** nodig om SCSS manueel te installeren via NPM zoals hierboven beschreven.

WEL doen

```

<script setup>
// Vue-component code hier
</script>

<template>

```

vue


```
<!-- Vue-template code hier -->
</template>

<style lang="scss">
// SCSS-code hier
</style>
```

In de rest van het opleidingsonderdeel zal **Vanilla CSS** gebruikt worden in Vue-componenten. Het gebruik van SCSS in Vue-componenten kan gebeuren via het **lang="scss"** -attribuut in de **<style>** -tag, maar dit is optioneel en niet verplicht. Ook zal er gebruik gemaakt worden van een component library die onderliggend SCSS gebruikt.

Conclusie

Het is belangrijk om te begrijpen wat SCSS is (een CSS preprocessor) en dat het extra functionaliteiten toevoegt aan gewone CSS, zoals variabelen, nested rules, mixins, functies, modules en imports. Je moet SCSS-code naar gewone CSS compileren voordat browsers deze kunnen begrijpen.

Veel concepten van preprocessors zijn doorheen de jaren toegevoegd aan gewone CSS, zoals CSS-variabelen (CSS Custom Properties) en nested rules.

[Previous page](#)

[Oefeningen](#)

